

DESIGN AND ANALYSIS OF ALGORITHMS

Kruskal's Algorithm

Finding a Minimum Spanning Tree using the Greedy Approach

Simrat Singh Arora

PRN: 25070521181

Section / Batch: D1

Generated using Claude (Anthropic AI)

Background: Minimum Spanning Tree

- Given a connected, weighted, undirected graph $G = (V, E)$
- A spanning tree connects all V vertices using exactly $V - 1$ edges, with no cycles
- A Minimum Spanning Tree (MST) is the spanning tree with the smallest total edge weight
- Goal of Kruskal's algorithm: build the MST edge by edge, always choosing the cheapest safe edge

$V - 1$ edges in an MST

0 cycles allowed

Min total weight

Key Idea of Kruskal's Algorithm

Greedy

Always pick the smallest-weight edge available

Safe

Skip any edge that would create a cycle

Forest

Starts as V separate trees; merges them into one

Correctness: by the cut property, the lightest edge crossing any cut of the graph belongs to some MST, so each greedy choice is safe.

Algorithm Steps

- 1 Sort all edges in non-decreasing order of weight
- 2 Create a disjoint set (Union-Find) with each vertex in its own set
- 3 Pick the next smallest edge (u, v)
- 4 If u and v are in different sets, add the edge to the MST and union the sets
- 5 Otherwise discard the edge (it forms a cycle)
- 6 Stop when the MST has $V - 1$ edges

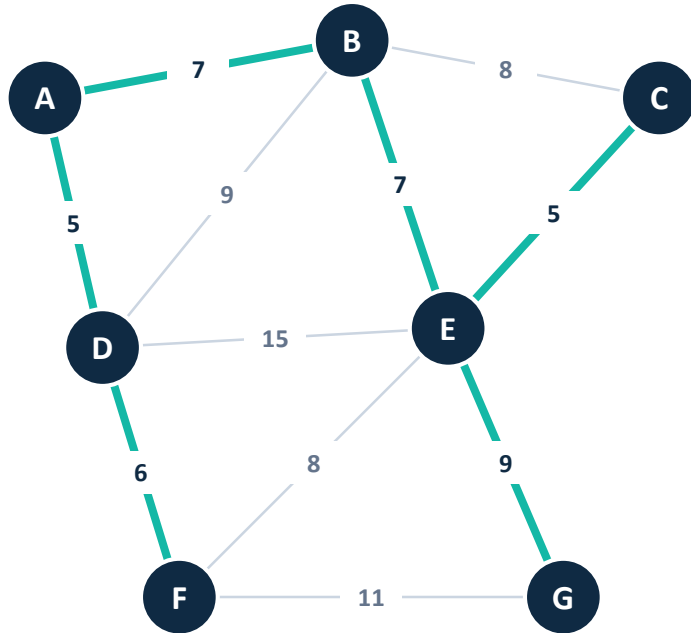
Pseudocode

```
KRUSKAL(G):  
  A = { }  
  for each vertex v in G.V:  
    MAKE-SET(v)  
  sort G.E by weight w  
  for each edge (u, v) in sorted G.E:  
    if FIND-SET(u) != FIND-SET(v):  
      A = A  $\cup$  { (u, v) }  
      UNION(u, v)  
  return A
```

Union-Find (Disjoint Set)

- FIND-SET: which component does a vertex belong to?
- UNION: merge two components
- Path compression and union by rank make operations nearly $O(1)$

Worked Example



#	Edge	Weight	Decision
1	A–D	5	Accept
2	C–E	5	Accept
3	D–F	6	Accept
4	A–B	7	Accept
5	B–E	7	Accept
6	B–C	8	Reject (cycle)
7	E–F	8	Reject (cycle)
8	B–D	9	Reject (cycle)
9	E–G	9	Accept

MST weight = $5 + 5 + 6 + 7 + 7 + 9 = 39$ (6 edges for 7 vertices)

Step-by-Step: Tracing the Union-Find Sets

Start: {A} {B} {C} {D} {E} {F} {G} (every vertex in its own set)

#1 A-D (5) **ACCEPT** {A,D} {B} {C} {E} {F} {G}

#2 C-E (5) **ACCEPT** {A,D} {C,E} {B} {F} {G}

#3 D-F (6) **ACCEPT** {A,D,F} {C,E} {B} {G}

#4 A-B (7) **ACCEPT** {A,B,D,F} {C,E} {G}

#5 B-E (7) **ACCEPT** {A,B,C,D,E,F} {G}

#6 B-C (8) **REJECT** both already in the big set → cycle, skip

#7 E-F (8) **REJECT** both already in the big set → cycle, skip

#8 B-D (9) **REJECT** both already in the big set → cycle, skip

#9 E-G (9) **ACCEPT** {A,B,C,D,E,F,G} → 6 edges, MST complete!

Why Are Some Edges Rejected?

B-C (8)

B and C are already connected through B-E and E-C.

Cycle: $B \rightarrow E \rightarrow C \rightarrow B$

E-F (8)

E and F are already connected through E-B, B-A, A-D and D-F.

Cycle: $E \rightarrow B \rightarrow A \rightarrow D \rightarrow F \rightarrow E$

B-D (9)

B and D are already connected through B-A and A-D.

Cycle: $B \rightarrow A \rightarrow D \rightarrow B$

Final MST: A-D, C-E, D-F, A-B, B-E, E-G Total weight = 39

Time and Space Complexity

TIME COMPLEXITY

$$O(E \log V)$$

SPACE COMPLEXITY

$$O(V + E)$$

Step	Cost
Sort the edges	$O(E \log E)$
V MAKE-SET operations	$O(V)$
E FIND / UNION operations	$O(E \cdot \alpha(V)) \approx O(E)$
Total time	$O(E \log E) = O(E \log V)$
Space (edges + sets)	$O(V + E)$

Why $E \log E = E \log V$?

Since $E \leq V^2$, $\log E \leq 2 \log V$, so both bounds are equivalent. Sorting dominates the running time.

Kruskal's vs Prim's Algorithm

Feature	Kruskal's	Prim's
Approach	Edge-based, greedy	Vertex-based, greedy
Starts from	Smallest edge overall	An arbitrary vertex
Intermediate result	Forest of trees	Single growing tree
Data structure	Union-Find + sorting	Priority queue / min-heap
Time	$O(E \log V)$	$O(E \log V)$ with heap
Best for	Sparse graphs	Dense graphs

Applications and Limitations

Applications

- Network design (LAN, telecom, cable layouts)
- Road, pipeline and electrical grid planning
- Clustering in machine learning
- Approximation for the Travelling Salesman Problem
- Image segmentation

Strengths and limits

- Simple to implement
- Efficient on sparse graphs
- Works on disconnected graphs (gives a minimum spanning forest)
- Needs all edges sorted up front
- Slower than Prim on very dense graphs

Conclusion

- Kruskal's algorithm is a classic greedy method for building an MST
- Sort edges, add the cheapest one that does not form a cycle
- Runs in $O(E \log V)$ time using Union-Find
- Ideal for sparse graphs and many real-world network problems

Thank you!

Simrat Singh Arora | PRN 25070521181 | Section/Batch D1
Generated using Claude (Anthropic AI)